Product Analytics SSL Troubleshooting

Note: For running the `kubectl` commands suggested in this document, you need to make sure you're in the correct kubernetes context and namespace. You can find out which context to run these commands by checking the "Cluster" dropdown in the monitoring dashboard. All commands should be run in the `default` namespace.

Services affected by SSL outages

All of our external endpoints are using SSL certificates, as well as the internal communication between various services. So an SSL outage could affect the entire stack if multiple certificates fail at the same time:

- Outages to the Snowplow collector will prevent events being ingested by the stack.
- Outages to the Cube endpoint or Clickhouse will prevent events from being retrievable.
- Outages to the configurator will prevent new projects from being onboarded to Product Analytics.

Internally, SSL outages to the Snowplow enricher, Vector, Kafka, or Clickhouse, will prevent events from being processed. Although, we do have queuing systems in place to help retain events in the situation that they can't be processed, it won't keep the events in a holding pattern forever.

Examples of SSL errors

We send SSL metrics to our monitoring dashboard if you would like to observe the errors over time. You can also review the various logs through that dashboard too.

There are few errors we've seen in the past which indicate that we're having an SSL outage. These include:

Vector

```
"2023-12-14T08:29:31.003191Z WARN sink{component_kind="sink" component_id=clickhouse_enriche
```

Snowplow enricher

```
Failed authentication with <REDACTED_CLUSTER_NAME>-kafka/<REDACTED_IP> (SSL handshake failed
```

```
 [pool-1-thread-2] INFO com.snowplowanalytics.snowplow.enrich.common.fs2.Environment - Enri
 org.apache.kafka.common.errors.SslAuthenticationException: SSL handshake failed
 Caused by: javax.net.ssl.SSLHandshakeException: PKIX path validation failed: java.security
```

Kafka

```
 [2023-12-14 10:14:23,906] INFO [SocketServer listenerType=ZK_BROKER, nodeId=0] Failed auth
```

Kafka exporter

`Cannot get current offset of topic __consumer_offsets partition 46: x509: certificate has ex`

LoadBalancer

GKE events from Ingress type can be found in this panel Alternatively, you can run `kubectl get events --field-selector involvedObject.kind=Ingress`

`Error syncing to GCP: error running backend syncing routine: error ensuring health check: go`

Fixing SSL errors

First, you should identify which certificates have failed. A good place to start is to view our monitoring dashboard and check for any telltale logs. If you can narrow down which services have been affected, this will help narrow down which certificates may be causing issues.

Once you know which services may be affected, you can use Kubernetes to read certificate details. You will need to follow the prerequisite steps for the Analytics Stack to be able to run these commands.

Start by getting all the certificates known by Kubernetes:

```
kubectl get certificates
```

This will give you a list of certificate names and how old they are. Oftentimes, the certificate renewal will have failed. The age of the certificate will give you an indication this may be the case.

For more details about a specific certificate, you can use the certificates name from the above command to get more information:

```
kubectl describe certificate <CERT_NAME>
```

Within the output, you will find a `Status` subsection which may show any problems:

```
Status:
  Conditions:
    Last Transition Time:  2023-09-07T08:38:37Z
    Message:               Certificate is up to date and has not expired
    Observed Generation:   1
    Reason:                Ready
    Status:                True
    Type:                  Ready
  Not After:               2024-04-04T08:38:36Z
  Not Before:              2024-01-05T08:38:36Z
  Renewal Time:            2024-03-05T08:38:36Z
  Revision:                3
```

You can also get a specific certificates expiry directly:

```
kubectl get secret <CLUSTER_NAME>-certificates-<SECRET_NAME> -o "jsonpath={.data['tls\\.crt'
```

We use the cert-manager Kubernetes plugin to manage SSL certificates, with LetsEncrypt certificates for external endpoints, and self-signed certificates for internal communications. This means we can use the `cmctl` to manually renew certificates as required.

Once the certificates have been created or renewed, you may need to redeploy to Kubernetes. The easiest way to do this is to use the existing CI pipelines.

1. Find the most recent release.
2. Click the commit at the bottom of the release.
3. Click the pipeline for the commit.
4. Trigger the environment deployment for the affected environment.

In some cases, certificates might have a status ready while the certificate issuer is not ready. You can use the following command to get information on issuers.

```
kubectl get issuer
```

Confirm that all issuers have True in the READY column. If not, get more information on the specific issuer with

```
kubectl describe issuer [issuer_name]
```

which will give you detailed information on the issuer. You can check the Status field and look for "Reason" and "Message". These fields should give information on why the issuer is not ready.

Kafka exporter

The Kafka exporter uses a self-signed certificate, which does not auto-renew. This can mean that the certificate expires if we haven't made any changes for a longer period of time.

If you're seeing Kafka export errors in Grafana, then the easy fix is to restart the deployment:

```
kubectl rollout restart deployment <CLUSTER_NAME>-kafka-exporter
```